

Algoritmos - Actividad Guiada 1

Nombre: Reda Bennor Haliba

URL: https://colab.research.google.com/drive/1EHu24iq_4SZhSy0hkFa9uD8rjsfOwupN?usp=sharing

Torres de Hanoi con Divide y vencerás

```
In [5]: def Torres_Hanoi(N, desde, hasta):
    if N ==1 :
        print("Lleva la ficha " ,desde , " hasta " , hasta )

    else:
        #Torres_Hanoi(N-1, desde, 6-desde-hasta )
        Torres_Hanoi(N-1, desde, 6-desde-hasta )
        print("Lleva la ficha " ,desde , " hasta " , hasta )
        #Torres_Hanoi(N-1,6-desde-hasta, hasta )
        Torres_Hanoi(N-1, 6-desde-hasta , hasta )

Torres_Hanoi(3, 1 , 3)
```

```
Lleva la ficha 1 hasta 3
Lleva la ficha 1 hasta 2
Lleva la ficha 3 hasta 2
Lleva la ficha 1 hasta 3
Lleva la ficha 2 hasta 1
Lleva la ficha 2 hasta 3
Lleva la ficha 1 hasta 3
```

```
In [6]: #Sucesión_de_Fibonacci
#https://es.wikipedia.org/wiki/Sucesi%C3%B3n_de_Fibonacci
#Calculo del termino n-simo de la sucesión de Fibonacci
def Fibonacci(N:int):
    if N < 2:
        return 1
    else:
        return Fibonacci(N-1)+Fibonacci(N-2)

Fibonacci(5)
```

Out[6]: 8

Devolución de cambio por técnica voraz

```
In [7]: def cambio_monedas(N, SM):
    SOLUCION = [0]*len(SM)    #SOLUCION = [0,0,0,0,..]
    ValorAcumulado = 0

    for i,valor in enumerate(SM):
        monedas = (N-ValorAcumulado)//valor
        SOLUCION[i] = monedas
```

```

ValorAcumulado = ValorAcumulado + monedas*valor

if ValorAcumulado == N:
    return SOLUCION

cambio_monedas(15,[25,10,5,1])

```

Out[7]: [0, 1, 1, 0]

N-Reinas por técnica de vueta atrás

```

In [8]: def escribe(S):
n = len(S)
for x in range(n):
    print("")
    for i in range(n):
        if S[i] == x+1:
            print(" X ", end="")
        else:
            print(" - ", end="")

def es_prometedora(SOLUCION,etapa):
    #print(SOLUCION)
    #Si la solución tiene dos valores iguales no es valida => Dos reinas en la mis
    for i in range(etapa+1):
        #print("El valor " + str(SOLUCION[i]) + " está " + str(SOLUCION.count(SOLUCION[i]))
        if SOLUCION.count(SOLUCION[i]) > 1:
            return False

        #Verifica las diagonales
        for j in range(i+1, etapa + 1 ):
            #print("Comprobando diagonal de " + str(i) + " y " + str(j))
            if abs(i-j) == abs(SOLUCION[i]-SOLUCION[j]) : return False
    return True

def reinas(N, solucion=[], etapa=0):
    if len(solucion) == 0:
        solucion=[0 for i in range(N)]

    for i in range(1, N+1):
        solucion[etapa] = i

        if es_prometedora(solucion, etapa):
            if etapa == N-1:
                print(solucion)
                #escribe(solucion)
                print()
            else:
                reinas(N, solucion, etapa+1)
        else:
            None

    solucion[etapa] = 0

```

```
reinas(8)
```

[1, 5, 8, 6, 3, 7, 2, 4]
[1, 6, 8, 3, 7, 4, 2, 5]
[1, 7, 4, 6, 8, 2, 5, 3]
[1, 7, 5, 8, 2, 4, 6, 3]
[2, 4, 6, 8, 3, 1, 7, 5]
[2, 5, 7, 1, 3, 8, 6, 4]
[2, 5, 7, 4, 1, 8, 6, 3]
[2, 6, 1, 7, 4, 8, 3, 5]
[2, 6, 8, 3, 1, 4, 7, 5]
[2, 7, 3, 6, 8, 5, 1, 4]
[2, 7, 5, 8, 1, 4, 6, 3]
[2, 8, 6, 1, 3, 5, 7, 4]
[3, 1, 7, 5, 8, 2, 4, 6]
[3, 5, 2, 8, 1, 7, 4, 6]
[3, 5, 2, 8, 6, 4, 7, 1]
[3, 5, 7, 1, 4, 2, 8, 6]
[3, 5, 8, 4, 1, 7, 2, 6]
[3, 6, 2, 5, 8, 1, 7, 4]
[3, 6, 2, 7, 1, 4, 8, 5]
[3, 6, 2, 7, 5, 1, 8, 4]
[3, 6, 4, 1, 8, 5, 7, 2]
[3, 6, 4, 2, 8, 5, 7, 1]
[3, 6, 8, 1, 4, 7, 5, 2]
[3, 6, 8, 1, 5, 7, 2, 4]
[3, 6, 8, 2, 4, 1, 7, 5]
[3, 7, 2, 8, 5, 1, 4, 6]
[3, 7, 2, 8, 6, 4, 1, 5]
[3, 8, 4, 7, 1, 6, 2, 5]
[4, 1, 5, 8, 2, 7, 3, 6]
[4, 1, 5, 8, 6, 3, 7, 2]

[4, 2, 5, 8, 6, 1, 3, 7]

[4, 2, 7, 3, 6, 8, 1, 5]

[4, 2, 7, 3, 6, 8, 5, 1]

[4, 2, 7, 5, 1, 8, 6, 3]

[4, 2, 8, 5, 7, 1, 3, 6]

[4, 2, 8, 6, 1, 3, 5, 7]

[4, 6, 1, 5, 2, 8, 3, 7]

[4, 6, 8, 2, 7, 1, 3, 5]

[4, 6, 8, 3, 1, 7, 5, 2]

[4, 7, 1, 8, 5, 2, 6, 3]

[4, 7, 3, 8, 2, 5, 1, 6]

[4, 7, 5, 2, 6, 1, 3, 8]

[4, 7, 5, 3, 1, 6, 8, 2]

[4, 8, 1, 3, 6, 2, 7, 5]

[4, 8, 1, 5, 7, 2, 6, 3]

[4, 8, 5, 3, 1, 7, 2, 6]

[5, 1, 4, 6, 8, 2, 7, 3]

[5, 1, 8, 4, 2, 7, 3, 6]

[5, 1, 8, 6, 3, 7, 2, 4]

[5, 2, 4, 6, 8, 3, 1, 7]

[5, 2, 4, 7, 3, 8, 6, 1]

[5, 2, 6, 1, 7, 4, 8, 3]

[5, 2, 8, 1, 4, 7, 3, 6]

[5, 3, 1, 6, 8, 2, 4, 7]

[5, 3, 1, 7, 2, 8, 6, 4]

[5, 3, 8, 4, 7, 1, 6, 2]

[5, 7, 1, 3, 8, 6, 4, 2]

[5, 7, 1, 4, 2, 8, 6, 3]

[5, 7, 2, 4, 8, 1, 3, 6]

[5, 7, 2, 6, 3, 1, 4, 8]

[5, 7, 2, 6, 3, 1, 8, 4]

[5, 7, 4, 1, 3, 8, 6, 2]

[5, 8, 4, 1, 3, 6, 2, 7]

[5, 8, 4, 1, 7, 2, 6, 3]

[6, 1, 5, 2, 8, 3, 7, 4]

[6, 2, 7, 1, 3, 5, 8, 4]

[6, 2, 7, 1, 4, 8, 5, 3]

[6, 3, 1, 7, 5, 8, 2, 4]

[6, 3, 1, 8, 4, 2, 7, 5]

[6, 3, 1, 8, 5, 2, 4, 7]

[6, 3, 5, 7, 1, 4, 2, 8]

[6, 3, 5, 8, 1, 4, 2, 7]

[6, 3, 7, 2, 4, 8, 1, 5]

[6, 3, 7, 2, 8, 5, 1, 4]

[6, 3, 7, 4, 1, 8, 2, 5]

[6, 4, 1, 5, 8, 2, 7, 3]

[6, 4, 2, 8, 5, 7, 1, 3]

[6, 4, 7, 1, 3, 5, 2, 8]

[6, 4, 7, 1, 8, 2, 5, 3]

[6, 8, 2, 4, 1, 7, 5, 3]

[7, 1, 3, 8, 6, 4, 2, 5]

[7, 2, 4, 1, 8, 5, 3, 6]

[7, 2, 6, 3, 1, 4, 8, 5]

[7, 3, 1, 6, 8, 5, 2, 4]

[7, 3, 8, 2, 5, 1, 6, 4]

[7, 4, 2, 5, 8, 1, 3, 6]

[7, 4, 2, 8, 6, 1, 3, 5]

[7, 5, 3, 1, 6, 8, 2, 4]

[8, 2, 4, 1, 7, 5, 3, 6]

[8, 2, 5, 3, 1, 7, 4, 6]

[8, 3, 1, 6, 2, 5, 7, 4]

[8, 4, 1, 3, 6, 2, 7, 5]

Viaje por el río. Programación dinámica

```
In [9]: TARIFAS = [
[0,5,4,3,999,999,999],
[999,0,999,2,3,999,11],
[999,999, 0,1,999,4,10],
[999,999,999, 0,5,6,9],
[999,999, 999,999,0,999,4],
[999,999, 999,999,999,0,3],
[999,999,999,999,999,999,0]
]

#####
def Precios(TARIFAS):
#####
    #Total de Nodos
    N = len(TARIFAS[0])

    #Inicialización de la tabla de precios
    PRECIOS = [ [9999]*N for i in range(9999)*N]
    RUTA = [ [""]*N for i in range(9999)*N]

    for i in range(0,N-1):
        RUTA[i][i] = i                #Para ir de i a i se "pasa por i"
        PRECIOS[i][i] = 0             #Para ir de i a i se se paga 0
        for j in range(i+1, N):
            MIN = TARIFAS[i][j]
            RUTA[i][j] = i

            for k in range(i, j):
                if PRECIOS[i][k] + TARIFAS[k][j] < MIN:
                    MIN = min(MIN, PRECIOS[i][k] + TARIFAS[k][j] )
                    RUTA[i][j] = k                #Anota que para ir de i a j hay que pasar po
                PRECIOS[i][j] = MIN

    return PRECIOS,RUTA
#####

PRECIOS,RUTA = Precios(TARIFAS)
#print(PRECIOS[0][6])

print("PRECIOS")
for i in range(len(TARIFAS)):
    print(PRECIOS[i])

print("\nRUTA")
for i in range(len(TARIFAS)):
    print(RUTA[i])

#Determinar la ruta con Recursividad
def calcular_ruta(RUTA, desde, hasta):
```

```

if desde == hasta:
    #print("Ir a :" + str(desde))
    return ""
else:
    return str(calcular_ruta( RUTA, desde, RUTA[desde][hasta])) + \
           ',' + \
           str(RUTA[desde][hasta] \
              )

print("\nLa ruta es:")
calcular_ruta(RUTA, 0,6)

```

PRECIOS

```

[0, 5, 4, 3, 8, 8, 11]
[9999, 0, 999, 2, 3, 8, 7]
[9999, 9999, 0, 1, 6, 4, 7]
[9999, 9999, 9999, 0, 5, 6, 9]
[9999, 9999, 9999, 9999, 0, 999, 4]
[9999, 9999, 9999, 9999, 9999, 0, 3]
[9999, 9999, 9999, 9999, 9999, 9999, 9999]

```

RUTA

```

[0, 0, 0, 0, 1, 2, 5]
['', 1, 1, 1, 1, 3, 4]
['', '', 2, 2, 3, 2, 5]
['', '', '', 3, 3, 3, 3]
['', '', '', '', 4, 4, 4]
['', '', '', '', '', 5, 5]
['', '', '', '', '', '', '']

```

La ruta es:

Out[9]: ',0,2,5'

Ordenacion por Mezcla (MergeSort) con Divide y Vencerás

```

In [10]: # Ordenacion por Mezcla (MergeSort)
# Divide la lista en mitades, ordena cada mitad recursivamente y las mezcla

def mezclar(izq, der):
    resultado = []
    i = j = 0

    while i < len(izq) and j < len(der):
        if izq[i] <= der[j]:
            resultado.append(izq[i])
            i += 1
        else:
            resultado.append(der[j])
            j += 1

    resultado.extend(izq[i:])
    resultado.extend(der[j:])
    return resultado

def merge_sort(lista):
    if len(lista) <= 1:

```



```

    return lista

    mitad = len(lista) // 2
    izquierda = merge_sort(lista[:mitad])
    derecha = merge_sort(lista[mitad:])

    return mezclar(izquierda, derecha)

lista = [38, 27, 43, 3, 9, 82, 10]
print("Lista original:", lista)
print("Lista ordenada:", merge_sort(lista))

```

Lista original: [38, 27, 43, 3, 9, 82, 10]
 Lista ordenada: [3, 9, 10, 27, 38, 43, 82]

Selección de Actividades por técnica voraz

```

In [11]: # Problema de selección de actividades
# Dadas actividades con hora de inicio y fin, seleccionar el máximo
# número de actividades que no se solapen
# Estrategia voraz: ordenar por hora de fin y elegir siempre la que termina antes

def seleccion_actividades(actividades):
    # Ordenar por hora de fin
    actividades_ord = sorted(actividades, key=lambda x: x[1])

    seleccionadas = [actividades_ord[0]]
    ultimo_fin = actividades_ord[0][1]

    for i in range(1, len(actividades_ord)):
        inicio, fin = actividades_ord[i]
        if inicio >= ultimo_fin: # No se solapa con la última seleccionada
            seleccionadas.append(actividades_ord[i])
            ultimo_fin = fin

    return seleccionadas

# Actividades como tuplas (inicio, fin)
actividades = [(1,4), (3,5), (0,6), (5,7), (3,9), (5,9), (6,10), (8,11), (8,12),
               (2,14), (12,16)]

resultado = seleccion_actividades(actividades)
print("Actividades disponibles:", actividades)
print("Actividades seleccionadas:", resultado)
print("Total de actividades compatibles:", len(resultado))

```

Actividades disponibles: [(1, 4), (3, 5), (0, 6), (5, 7), (3, 9), (5, 9), (6, 10), (8, 11), (8, 12), (2, 14), (12, 16)]
 Actividades seleccionadas: [(1, 4), (5, 7), (8, 11), (12, 16)]
 Total de actividades compatibles: 4

Suma de Subconjuntos por técnica de vuelta atrás

```

In [12]: # Problema de la suma de subconjuntos
# Dado un conjunto de números, encontrar todos los subconjuntos que sumen un valor
# Se usa backtracking: se construye el subconjunto elemento a elemento
# y se poda cuando la suma parcial supera el objetivo

```

```

def suma_subconjuntos(conjunto, objetivo, parcial=[], inicio=0):
    suma_actual = sum(parcial)

    # Caso base: encontramos una solucion
    if suma_actual == objetivo:
        print(parcial)
        return

    # Poda: si la suma parcial ya supera el objetivo, no seguimos
    if suma_actual > objetivo:
        return

    # Probar con cada elemento restante
    for i in range(inicio, len(conjunto)):
        suma_subconjuntos(conjunto, objetivo, parcial + [conjunto[i]], i + 1)

conjunto = [3, 7, 1, 8, 4, 12, 5]
objetivo = 12
print(f"Subconjuntos de {conjunto} que suman {objetivo}:")
suma_subconjuntos(conjunto, objetivo)

```

Subconjuntos de [3, 7, 1, 8, 4, 12, 5] que suman 12:

```

[3, 1, 8]
[3, 4, 5]
[7, 1, 4]
[7, 5]
[8, 4]
[12]

```